

Algoritmos y Estructuras de Datos III

Trabajo Práctico N° 6	Unidad 4
Modalidad: Semi -Presencial	Estratégica Didáctica: Trabajo individual
Metodología de Desarrollo: Det. docente	Metodología de Corrección: Via Classroom.
Carácter de Trabajo: Obligatorio – Con Nota	Fecha Entrega: A confirmar por el Docente.

GRAFOS

MARCO TEÓRICO:

Responder el siguiente cuestionario en función de la bibliografía Obligatoria.

1. ¿Qué es un Grafo?

Un grafo es una herramienta que se utiliza para agrupar objetos y mostrar las relaciones entre ellos. Está formado por un conjunto de puntos llamados vértices o nodos, que representan los objetos, y un conjunto de líneas llamadas arcos, que representan las relaciones entre esos puntos.

Un grafo se puede representar como un par de conjuntos: $G = (V, A)$, donde V es el conjunto de vértices y A es el conjunto de arcos.

Hay dos tipos de grafos:

- Los grafos no dirigidos, donde las líneas entre los puntos no tienen dirección y se representan con un segmento.
- Los grafos dirigidos, también llamados digrafos, donde las líneas entre los puntos tienen una dirección específica y se representan con una flecha.

Cuando una línea entre dos puntos tiene un valor numérico asociado, se habla de un grafo valorado o ponderado.

Los grafos tienen muchas aplicaciones en diferentes campos, como:

- Sociología
- Química
- Geografía
- Ingeniería eléctrica
- Planificación de proyectos
- Rutas de menor longitud
- Flujo máximo en redes

En general, los grafos son una herramienta muy útil para analizar y visualizar relaciones entre objetos en diversas áreas.

2. Explicar le concepto de vértice y Arco

Un vértice es cada uno de los elementos que forman un grafo. Esto puede ser algo como una ciudad, una persona o un proceso. Los vértices se numeran desde cero para que sea más fácil encontrarlos en la memoria.

Un arco une dos vértices. Se escribe como el par de vértices que conecta. Si dos vértices están unidos por un arco, se dice que son adyacentes. En un grafo donde se tiene en cuenta la dirección, un vértice puede estar conectado a otro, pero no al revés.

El grado de un vértice en un grafo donde no se tiene en cuenta la dirección es el número de arcos que lo tocan. En un grafo donde se tiene en cuenta la dirección, hay dos tipos de grados: el grado de entrada, que es el número de arcos que llegan al vértice, y el grado de salida, que es el número de arcos que salen de él.

Un camino es una serie de vértices que van desde un vértice hasta otro. La longitud del camino es el número de arcos que se necesitan para ir de un vértice a otro. Un camino es simple si no se repiten vértices. Un ciclo es un camino que empieza y termina en el mismo vértice, sin repetir ningún otro vértice.

3. ¿Como se Representa un Grafo?, en qué casos son más útiles las opciones mejoradas?

Hay dos formas principales de guardar un grafo en la memoria de una computadora.

Matriz de adyacencia

Es una tabla de $n \times n$ casillas (donde n son los vértices del grafo). Cada casilla **a_{ij}** es 1 si hay un arco del vértice **i** al **j** , o 0 si no lo hay. Si el grafo tiene pesos, la casilla **a_{ij}** tiene el valor del peso.

Esta forma de guardar el grafo es compacta. Puedes saber si dos vértices están conectados en un abrir y cerrar de ojos ($O(1)$). Pero usa mucha memoria ($O(n^2)$), da igual si hay muchos o pocos arcos.

Listas de adyacencia

Es una forma dinámica de guardar el grafo. Para cada vértice, hay una lista con los vértices que están conectados a él.

Esta forma usa memoria según los vértices y arcos que hay ($O(V + A)$).

La elección depende del tipo de grafo y de lo que quieras hacer con él. La matriz es mejor para grafos densos (con muchos arcos), porque merece la pena usar más memoria por el acceso rápido. Las listas son mejores para grafos dispersos (con pocos arcos), porque ahorran memoria.

4. Como se recorre un grafo, enumerar los pasos de estos.

Recorrido en anchura (BFS)

Esta estrategia utiliza una cola como estructura auxiliar. El proceso es el siguiente:

Se marca el vértice de partida v .

Se agrega v a la cola.

Se repiten los pasos 1 y 2 hasta que la cola esté vacía.

1. Se saca el nodo que está al frente de la cola, w , y se visita.
2. Se agregan a la cola todos los vértices adyacentes a w que no estén marcados y se marcan.

Recorrido en profundidad (DFS)

Esta estrategia utiliza una pila como estructura auxiliar. El proceso es el siguiente:

Se marca el vértice de partida v .

Se agrega v a la pila.

Se repiten los pasos 1 y 2 hasta que la pila esté vacía.

1. Se saca el nodo que está en la cabeza de la pila, w , y se visita.
2. Se agregan a la pila todos los vértices adyacentes a w que no estén marcados y se marcan.

La principal diferencia entre ambos es el orden en que visitan los vértices. El recorrido en anchura (BFS) visita los vértices por niveles de distancia al origen, es decir, primero visita todos los vértices que están a una distancia de 1 del vértice de partida, luego los que están a una distancia de 2, y así sucesivamente.

Por otro lado, el recorrido en profundidad (DFS) sigue un camino hasta el final antes de explorar otras ramas. Esto significa que DFS avanza todo lo que puede hacia adelante antes de retroceder.

5. ¿Qué entiende por grafico Conexo?

Un grafo no dirigido es conexo cuando hay un camino que une cualquier par de vértices. Esto significa que se puede ir de un nodo a cualquier otro nodo del grafo. A estos grafos también se les llama redes conectadas.

En el caso de un grafo dirigido, la idea es similar, pero se considera la dirección de los arcos. Se dice que es fuertemente conexo cuando hay un camino entre cualquier par de vértices, siguiendo la dirección de los arcos.

Si un grafo no es conexo, se pueden encontrar sus componentes conexas. Estas son partes del grafo donde todos los vértices están conectados entre sí, pero no con el resto del grafo. Para encontrar estas componentes, se puede empezar en cualquier vértice y explorar el grafo en anchura o en profundidad. Los nodos que se pueden alcanzar forman una componente conexa. Si quedan nodos que no se han visitado, se elige uno de ellos y se repite el proceso hasta que todos los nodos hayan sido marcados.

6. ¿Qué es una Matriz de Caminos?

La matriz de caminos, también conocida como cierre transitivo, es una matriz de tamaño $n \times n$ que muestra si hay un camino entre dos vértices en un grafo. Cada elemento $P[i][j]$ de la matriz es 1 si existe un camino desde el vértice i hasta el vértice j , y 0 si no hay camino.

A diferencia de la matriz de adyacencia, que solo indica si hay un arco directo entre dos nodos, la matriz de caminos muestra si hay conectividad a través de cualquier cantidad de arcos intermedios.

Para construir esta matriz, se utiliza el algoritmo de Warshall. Este algoritmo comienza con la matriz de adyacencia y luego incorpora progresivamente cada vértice del grafo como posible intermediario de los caminos. De esta manera, se puede determinar si hay un camino entre dos vértices, aunque no sea directo.

7. ¿Cuál es el concepto detrás de cierre transitivo?, dar ejemplo.

El cierre transitivo de un grafo es como un mapa que muestra cómo se pueden conectar todos los puntos. Este mapa se llama G^* . Tiene los mismos puntos que el grafo original, pero muestra más conexiones.

Por ejemplo, imagina que tienes un grafo con cuatro puntos: A, B, C y D. Los arcos van de A a B, de B a C y de C a D. La matriz de adyacencia solo muestra estas conexiones directas. Pero, si miras más de cerca, verás que también hay conexiones indirectas.

Puedes ir de A a C y de A a D, aunque no haya arcos directos entre ellos. Lo mismo pasa con B y D.

El cierre transitivo agrega estas conexiones indirectas. Entonces, en la matriz de caminos resultante, verás una marca en todos los lugares donde hay una conexión, ya sea directa o indirecta. Esto nos dice si podemos llegar de un punto a otro siguiendo los arcos, sin importar cuántos saltos debamos dar.

8. ¿Dónde aplicaría ordenamiento topológico?

El ordenamiento topológico es una forma de organizar los vértices de un grafo dirigido acíclico de manera lineal. Esto significa que si hay un arco que va de un vértice u a un vértice v , entonces u debe aparecer antes que v en la ordenación. Esto solo funciona en grafos que no tienen ciclos.

Se utiliza en situaciones donde hay tareas que dependen de otras. Por ejemplo:

- + Planificación de proyectos: se ordenan las tareas de un proyecto para que se respeten las dependencias entre ellas. Una tarea no puede empezar hasta que no se hayan completado todas las tareas que la preceden.
- + Compiladores: se determina el orden en que se compilarán los módulos que tienen dependencias entre sí.
- + Gestión de paquetes de software: se instalan las dependencias antes de instalar el paquete que las necesita.
- + Cursos académicos: se ordenan las materias que tienen correlatividades, de modo que no se pueda cursar una materia sin haber aprobado las materias previas.
- + Procesamiento de hojas de cálculo: se evalúan las celdas que contienen fórmulas que dependen de otras celdas.

El algoritmo para realizar el ordenamiento topológico se basa en calcular el grado de entrada de cada nodo. Los nodos que no tienen dependencias, es decir, aquellos con un grado de entrada de 0, se procesan primero. Luego, cada vez que se procesa un nodo, se reduce el grado de entrada de los nodos que dependen de él. Cuando el grado de entrada de un nodo llega a 0, se agrega a la cola de nodos que se van a procesar.

9. ¿Qué usos tiene el algoritmo de WARSHALL?

El algoritmo de Warshall se utiliza para calcular la matriz de caminos en un grafo. Este grafo tiene n vértices y se parte de su matriz de adyacencia. El algoritmo de Warshall utiliza programación dinámica.

Esto significa que se definen n matrices, que van desde P_0 hasta P_n . Cada matriz P_k incorpora al vértice k como posible intermediario de los caminos. La matriz final, P_n , es el cierre transitivo del grafo.

El algoritmo de Warshall tiene varios usos. Puede determinar si existe algún camino entre cualquier par de vértices de un grafo. También puede encontrar componentes conexas y fuertemente conexas. Además, puede verificar si un grafo es conexo o fuertemente conexo.

Otro uso importante es resolver problemas de alcanzabilidad en redes de comunicación. Por ejemplo, puede determinar si un mensaje del nodo A puede llegar al nodo B. El algoritmo de Warshall también puede detectar relaciones transitivas en bases de datos y sistemas de inferencia.

La complejidad del algoritmo de Warshall es $O(n^3)$. Esto significa que es práctico para grafos de tamaño moderado.

10. Dar ejemplos de utilización del Algoritmo de DIJKSTRA

El algoritmo de Dijkstra encuentra el camino más corto desde un punto de partida hasta todos los demás puntos en un mapa con direcciones y distancias. Funciona de manera paso a paso, eligiendo siempre el siguiente punto que esté más cerca.

Este algoritmo se usa en:

- + Sistemas de navegación GPS: para encontrar la ruta más rápida o más corta entre dos lugares, teniendo en cuenta la distancia o el tiempo que tardas en recorrer cada parte del camino.
- + Redes de telecomunicaciones: para encontrar el camino con menos demora entre dos puntos de una red. Un ejemplo es el protocolo de enrutamiento OSPF que usa Dijkstra.
- + Logística: para calcular la ruta más económica para un vendedor que tiene que visitar varias ciudades.
- + Videojuegos: para encontrar el camino más fácil para que un personaje se mueva por un mapa con diferentes tipos de terreno.
- + Transporte público: para encontrar la combinación de líneas que hace que el viaje sea lo más corto posible entre dos paradas.

11. ¿Que es un Árbol de Expansión?

Un árbol de expansión de un grafo es básicamente un subconjunto del grafo que incluye todos los vértices y está bien conectado, pero no tiene ciclos. En un árbol con n vértices, siempre hay exactamente $n-1$ aristas. Si agregamos una arista a un árbol, se crea un ciclo, y si eliminamos una arista, el árbol ya no está conectado.

El problema del árbol de expansión de coste mínimo es encontrar el árbol de expansión con la suma de pesos de aristas más baja posible en un grafo no dirigido y ponderado que esté conectado. Esto es muy útil en el diseño de redes de comunicación, como cables de fibra óptica, tuberías y rutas eléctricas, donde el objetivo es conectar todos los nodos de la mejor manera posible para minimizar el costo total.

12. Que diferencias tienen los algoritmos de PRIM y KRUSCAL

Los algoritmos de Prim y Kruskal resuelven el mismo problema: encontrar el árbol de expansión de coste mínimo. Sin embargo, lo hacen de maneras diferentes.

El algoritmo de Prim comienza con un vértice inicial y va agregando vértices uno a uno. En cada paso, elige el vértice que no está en el árbol y que tiene la arista de menor coste con algún vértice que ya está en el árbol. De esta manera, nunca se forma un ciclo. El proceso continúa hasta que todos los vértices estén en el árbol. Este algoritmo es más rápido en grafos densos porque solo considera los vértices que ya están en el árbol.

Por otro lado, el algoritmo de Kruskal comienza con todos los vértices sin aristas, es decir, cada vértice es una componente por sí solo. Luego, ordena todas las aristas del grafo por su peso y las examina una a una. Si una arista conecta dos componentes diferentes, la agrega al árbol y fusiona esas dos componentes. Si la arista conecta vértices que ya están en la misma componente, la ignora porque agregarla formaría un ciclo. El proceso termina cuando todos los vértices están en la misma componente. Este algoritmo es más rápido en grafos dispersos porque considera todas las aristas del grafo.

La principal diferencia entre estos dos algoritmos es cómo construyen el árbol de expansión de coste mínimo. El algoritmo de Prim construye un árbol que crece desde un vértice inicial, mientras que el algoritmo de Kruskal construye un bosque de componentes que se van fusionando hasta formar un solo árbol.

13. Hacer un Resumen de las Utilidades de los Grafos en Memoria.

Los grafos en memoria son muy útiles para resolver problemas del mundo real.

Hay dos formas principales de representarlos: la matriz de adyacencia y las listas de adyacencia. La matriz de adyacencia es buena para grafos densos, porque puedes ver si dos nodos están conectados muy rápido. Las listas de adyacencia son mejores para grafos dispersos, porque solo guardan la información necesaria.

Para recorrer un grafo, hay dos estrategias. La primera es el recorrido en anchura, que explora los nodos nivel por nivel. Esto es útil para encontrar el camino más corto entre dos nodos. La segunda es el recorrido en profundidad, que avanza hacia adelante todo lo que pueda. Esto es útil para encontrar ciclos y explorar componentes.

Los algoritmos de recorrido también ayudan a determinar si un grafo está conectado o no. Si no lo está, puedes encontrar las partes conectadas. También puedes ver si hay algún camino entre dos nodos, sin importar la longitud del camino.

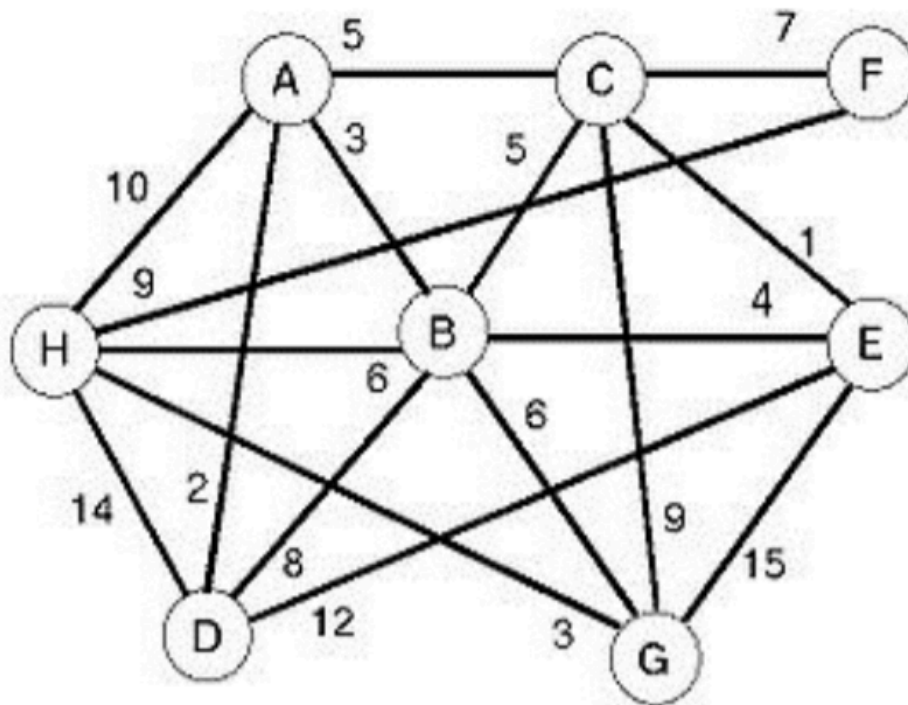
La ordenación topológica es útil para grafos dirigidos sin ciclos. Permite ordenar los nodos de manera que se respeten las dependencias. Esto es útil en planificación de proyectos, compiladores y gestión de paquetes de software.

Para encontrar los caminos más cortos, hay dos algoritmos útiles. El algoritmo de Dijkstra encuentra el camino más corto desde un nodo a todos los demás nodos. El algoritmo de Floyd encuentra el camino más corto entre todos los pares de nodos.

Finalmente, para diseñar redes con el menor costo posible, hay dos algoritmos útiles. Los algoritmos de Prim y Kruskal permiten conectar todos los nodos con el menor costo total. Esto es útil en redes de comunicación, tendido eléctrico y diseño de infraestructura.

MARCO PRÁCTICO

1. Dado el siguiente itinerario del recorrido de nuestro Agente de ventas que Muestra ciudades y costos asociados a, implementar el camino d emenor coste entre los nodos. Usar Dijkstra o Kruscal



Pd. Se adjuntan las soluciones “para referencia de Código”.

Se recomienda usarlo como referencia

https://github.com/FernandezPegoRocio/-Algoritmos_y_Estructuras-de-Datos_III

La Cátedra.
Lic. Oemig José Luis.